A CAMADA DE INFORMAÇÕES

2 - Valores Binários e Sistemas Numéricos

Agora que estabelecemos a história e alguma terminologia comum no Capítulo 1, nossa exploração da tecnologia da computação pode começar a sério. Este capítulo descreve os valores binários – a maneira como o hardware do computador representa e gerencia informações. Este capítulo também coloca os valores binários no contexto de todos os sistemas numéricos, lembrando-nos de conceitos do ensino fundamental que hoje consideramos óbvios. Você provavelmente já conhece muitos dos conceitos sobre números binários descritos neste capítulo, mas pode não perceber que os conhece! As regras de todos os sistemas numéricos são as mesmas; é apenas uma questão de retornar a esses conceitos subjacentes e aplicá-los em uma nova base. Ao garantir que temos uma compreensão dos valores binários, pavimentamos o caminho para a compreensão de como os sistemas de computação usam o sistema numérico binário para realizar suas tarefas.

OBJETIVOS

Após estudar este capítulo, você deverá ser capaz de:

- distinguir entre categorias de números.
- descrever a notação posicional.
- converter números de outras bases para a base 10.
- converter números de base 10 para números de outras bases.
- descrever a relação entre as bases 2, 8 e 16.
- explicar a importância para o cálculo de bases que são potências de 2.

2.1 Números e Computação

Os números são cruciais para a computação. Além de usar um computador para executar cálculos numéricos, todos os tipos de informação que armazenamos e gerenciamos usando um computador são, em última análise, armazenados como números. No nível mais baixo, os computadores armazenam todas as informações usando apenas os dígitos 0 e 1. Portanto, para começar nossa exploração dos computadores, precisamos primeiro explorar os números.

Primeiro, vamos lembrar que os números podem ser classificados em todos os tipos de categorias. Existem números naturais, números negativos, números reais, números racionais, números irracionais e muitos outros que são importantes em matemática, mas não para a compreensão da computação. Vamos revisar brevemente as definições de categorias relevantes.

Primeiro, vamos definir o conceito geral de um número: Um número é uma unidade pertencente a um sistema matemático abstrato e está sujeito a leis específicas de sucessão, adição e multiplicação. Ou seja, um número é uma representação de um valor, e certas operações aritméticas podem ser aplicadas consistentemente a esses valores.

* *Número*: Uma unidade de um sistema matemático abstrato sujeito às leis da aritmética.

Agora, vamos separar os números em categorias. Um número natural é o número 0 ou qualquer número obtido pela adição repetida de 1 a esse número. Números naturais são aqueles que usamos na contagem. Um número negativo é menor que zero e tem sinal oposto a um número positivo. Um número inteiro é qualquer um dos números naturais ou qualquer um dos negativos desses números.

Um número racional é um número inteiro ou o quociente de dois inteiros – ou seja, qualquer valor que pode ser expresso como uma fração.

- Número natural: O número 0 e qualquer número obtido pela adição repetida de 1 a ele.
- Número negativo: Um valor menor que 0, com sinal oposto ao seu equivalente positivo.
- Número inteiro: Um número natural, um negativo de um número natural ou zero.
- Número racional: Um número inteiro ou o quociente de dois inteiros (excluindo a divisão por zero).

Neste capítulo, focamos nos números naturais e nas maneiras como eles são representados em vários sistemas numéricos. Como parte da nossa discussão, estabelecemos como todos os sistemas numéricos se relacionam entre si. No Capítulo 3, examinamos a representação computacional de números negativos e racionais, bem como a forma como usamos números para representar outras formas de dados, como caracteres e imagens.

Parte do material deste capítulo pode já ser familiar para você. Certamente, algumas das ideias subjacentes devem ser. Você provavelmente não dá valor a alguns princípios básicos de números e aritmética porque se acostumou a eles. Parte do nosso objetivo neste capítulo é lembrá-lo desses princípios subjacentes e mostrar que eles se aplicam a todos os sistemas numéricos. Então, a ideia de que um computador usa valores binários – ou seja, 0s e 1s – para representar informações deve ser menos misteriosa.

2.2 Notação Posicional

Quantas unidades existem no número 943? Ou seja, quantos "pedacinhos" o número 943 representa, realmente? Bem, em termos do ensino fundamental, você poderia dizer que há 9 centenas mais 4 dezenas mais 3 unidades. Ou, dito de outra forma, há 900 unidades mais 40 unidades mais 3 unidades. Então, quantas unidades existem no número 754? 700 unidades mais 50 unidades mais 4 unidades. Certo? Bem, talvez. A resposta depende da base do sistema numérico que você está usando. Esta resposta está correta no sistema numérico de base 10, ou decimal, que é o sistema numérico que os humanos usam todos os dias. Mas essa resposta não está correta em outros sistemas numéricos, por exemplo, no octal.

A base de um sistema numérico especifica o número de dígitos usados no sistema. Os dígitos sempre começam com 0 e continuam até n – 1 (ou seja, um a menos que a base n). Por exemplo, há 2 dígitos na base 2: 0 e 1. Há 8 dígitos na base 8: 0 a 7. Há 10 dígitos na base 10: 0 a 9. A base também determina o significado das posições dos dígitos. Quando você adiciona 1 ao último dígito no sistema numérico, você tem um transporte para a posição do dígito à esquerda.

> * *Base*: É o valor fundamental de um sistema numérico, que determina o número de dígitos e o valor das posições dos dígitos.

Os números são escritos usando a notação posicional. O dígito mais à direita representa seu valor multiplicado pela base elevada à potência zero. O dígito à esquerda deste representa seu valor multiplicado pela base elevada à primeira potência. O próximo dígito representa seu valor multiplicado pela base elevada à segunda potência. O próximo dígito representa seu valor multiplicado pela base elevada à terceira potência, e assim por diante. Você está tão familiarizado com a notação posicional que provavelmente nem pensa nisso. Nós a usamos instintivamente para calcular o número de unidades em 943.

$$\begin{aligned} 9 * 10^2 &= 9 * 100 = 900 \\ + 4 * 10^1 &= 4 * 10 = 40 \\ + 3 * 10^0 &= 3 * 1 = 3 \\ &\qquad\qquad\quad 943 \end{aligned}$$

> * *Notação posicional*: um sistema de expressão numérica em que os dígitos são organizados em sucessão, a posição de cada dígito tem um valor posicional, e o número é igual à soma dos produtos de cada dígito por seu valor posicional[1].

? A importância do zero

A notação posicional só é possível graças ao conceito de zero. O zero foi o conceito fundamental na intersecção de todos os ramos da matemática moderna. Como Georges Ifrah observou em seu livro *The Universal History of Computing* (A História Universal da Computação): "Em resumo, a descoberta vital do zero deu à mente humana um potencial extraordinariamente poderoso. Nenhuma outra criação humana exerceu tamanha influência no desenvolvimento da inteligência da humanidade."[2]

Uma maneira mais formal de definir a notação posicional é dizer que o valor é representado como um polinômio na base do sistema numérico. Mas o que é um polinômio? Um polinômio é a soma de dois ou mais termos algébricos, cada um dos quais consiste em uma constante multiplicada por uma ou mais variáveis elevadas a uma potência integral não negativa. Ao definir a notação posicional a variável é a base do sistema numérico. Assim, 943 é representado como um polinômio da seguinte forma, com x atuando como a base:

$$9 * x^2 + 4 * x^1 + 3 * x^0$$

Vamos expressar essa ideia formalmente. Se um número no sistema numérico de base R tem n dígitos, ele é representado da seguinte forma, onde d_i representa o dígito na i-ésima posição do número:

$$d_n * R^{n-1} + d_{n-1} * R^{n-2} + d_{n-2} * R^{n-3} + \ldots + d_2 * R^1 + d_1$$

Parece complicado? Vejamos um exemplo concreto: 63578 na base 10. Aqui, n é 5 (o número tem cinco dígitos) e R é 10 (a base). A fórmula diz que o quinto dígito (último dígito à esquerda) é multiplicado pela base elevada à quarta potência; o quarto dígito é multiplicado pela base elevada à terceira potência; o terceiro dígito é multiplicado pela base elevada à segunda potência; o segundo dígito é multiplicado pela base elevada à primeira potência; e o primeiro dígito não é multiplicado por nada (na verdade, pela base elevada à 0).

$$6 * 10^4 + 3 * 10^3 + 5 * 10^2 + 7 * 10^1 + 8$$

No cálculo anterior, assumimos que a base do número é 10. Esta é uma suposição lógica, porque nosso sistema numérico é de base 10. No entanto, não há razão para que o número 943 não possa representar um valor na base 13. Se for o caso, para determinar o número de unidades, teríamos que convertê-lo para a base 10.

Portanto, 943 na base 13 é igual a 1576 na base 10. Lembre-se de que esses dois números têm um valor equivalente. Ou seja, ambos representam o mesmo número de "pedacinhos". Se um saco contém 943 (base 13) grãos de feijão e um segundo saco contém 1576 (base 10) grãos de feijão, então ambos os sacos contêm exatamente a mesma quantidade de grãos de feijão. Os sistemas numéricos nos permitem representar valores de várias maneiras.

Observe que na base 10, o dígito mais à direita é a posição das "unidades". Na base 13, o dígito mais à direita também é a posição das "unidades". Na verdade, isso é verdade para qualquer base, porque qualquer coisa elevada à potência 0 é 1.

Por que alguém iria querer representar valores na base 13? Não é feito com muita frequência, claro, mas às vezes é útil entender como funciona. Por exemplo, uma técnica de computação chamada *hashing* pega números e os embaralha, e uma maneira de embaralhar números é interpretá-los em uma base diferente.

Outras bases, como a base 2 (binária), são particularmente importantes no processamento computacional. Vamos explorar essas bases com mais detalhes.

Binário, Octal e Hexadecimal

O sistema numérico de base 2 (binário) é particularmente importante na computação. Também é útil estar familiarizado com sistemas numéricos que são potências de 2, como base 8 (octal) e base 16 (hexadecimal). Lembre-se de que o valor da base especifica o número de dígitos no sistema numérico. A base 10 tem dez dígitos (0-9), a base 2 tem dois dígitos (0-1) e a base 8 tem oito dígitos (0-7). Portanto, o número 943 não poderia representar um valor em nenhuma base menor que a base 10, porque o dígito 9 não existe nessas bases. No entanto, é um número válido na base 10 ou em qualquer base superior a essa, por exemplo, base 13 ou 16. Da mesma forma, o número 2074 é um número válido na base 8 ou superior, mas simplesmente não existe (porque usa o dígito 7) em nenhuma base inferior que essa.

Quais são os dígitos em bases maiores que 10? Precisamos de símbolos para representar os dígitos que correspondem aos valores decimais de 10 em diante. Em bases maiores que 10, usamos letras como dígitos. Usamos a letra A para representar o número 10, B para representar 11, C para representar 12 e assim por diante. Portanto, os 16 dígitos na base 16 são: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

Vamos analisar os valores em octal, hexadecimal e binário para ver o que eles representam na base 10. Por exemplo, vamos calcular o equivalente decimal de 754 em octal (base 8). Como antes, basta expandir o número em sua forma polinomial e somar os números.

$$\begin{aligned} 7 * 8^2 &= 7 * 64 = 448 \\ + 5 * 8^1 &= 5 * 8 = 40 \\ + 4 * 8^0 &= 4 * 1 = 4 \\ &\quad\quad\quad\quad 492 \end{aligned}$$

Vamos converter o número hexadecimal ABC para decimal:

$$
\begin{array}{rrrr}
A * 16^2 = & 10 * & 256 = & 2560 \\
+ B * 16^1 = & 11 * & 16 = & 176 \\
+ C * 16^0 = & 12 * & 1 = & 12 \\
 & & & \overline{2748}
\end{array}
$$

Observe que realizamos exatamente o mesmo cálculo para converter o número para a base 10. Usamos apenas um valor de base de 16 desta vez, e precisamos lembrar o que os dígitos das letras representam. Depois de um pouco de prática, você não achará o uso de letras como dígitos tão estranho.

Finalmente, vamos converter um número binário (base 2) 1010110 para decimal. Novamente, realizamos os mesmos passos – apenas o valor de base muda:

$$
\begin{array}{rrrr}
1 * 2^6 = & 1 * & 64 = & 64 \\
0 * 2^5 = & 0 * & 32 = & 0 \\
1 * 2^4 = & 1 * & 16 = & 16 \\
0 * 2^3 = & 0 * & 8 = & 0 \\
1 * 2^2 = & 1 * & 4 = & 4 \\
+ 1 * 2^1 = & 1 * & 2 = & 2 \\
+ 0 * 2^0 = & 0 * & 1 = & 0 \\
 & & & \overline{86}
\end{array}
$$

O Ábaco



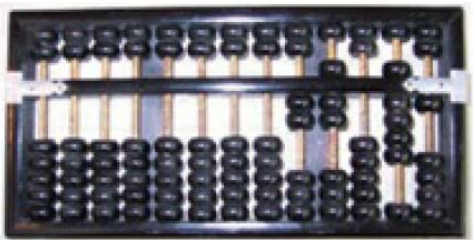
Cortesia de Theresa DiDonato

Em nossa breve história da computação no Capítulo 1, mencionamos o ábaco como um dos primeiros dispositivos de computação. Mais especificamente, o ábaco é um dispositivo que usa notação posicional para representar um número decimal. As contas em qualquer coluna representam o dígito dessa coluna. Todas as colunas combinadas representam um número completo.

As contas acima da barra central representam unidades de 5 e as contas abaixo da barra representam 1 cada. Contas afastadas da barra central não contribuem para o número.

A figura ao lado mostra o número 27.091 representado em um ábaco.

O usuário realiza cálculos movendo as contas de maneiras específicas para refletir as operações aritméticas básicas de adição, subtração, multiplicação e divisão.



Cortesia de Theresa DiDonato

Embora antigo, o ábaco ainda é usado hoje em muitas culturas asiáticas. Em lojas, um caixa pode usar um ábaco em vez de uma caixa registradora eletrônica. Embora não tenha algumas das vantagens dos dispositivos eletrônicos, o ábaco é mais do que suficiente para os tipos de cálculos necessários para transações comerciais básicas. Usuários habilidosos de um ábaco podem rivalizar com qualquer pessoa com uma calculadora em termos de velocidade e precisão.

As crianças nessas culturas aprendem operações de cor no ábaco, assim como você foi treinado em suas tabuadas de multiplicação. Para realizar uma operação com um número, o usuário executa uma série de movimentos usando apenas o polegar, o dedo indicador e o dedo médio de uma mão. Esses movimentos correspondem a dígitos individuais e dependem da operação que está sendo realizada. Por exemplo, para adicionar o dígito 7 ao dígito 5 já mostrado no ábaco, o usuário limpa o marcador de cinco (empurra-o para cima), empurra 2 na barra de baixo para cima e incrementa 1 na próxima coluna. Embora esse movimento corresponda à operação básica de adição que fazemos no papel, o usuário do ábaco não está pensando na matemática. O usuário é condicionado a executar um movimento específico quando dígitos específicos são encontrados para uma operação específica. Quando o cálculo é concluído, o usuário lê o resultado conforme mostrado no ábaco.

Lembre-se que os dígitos em qualquer sistema numérico vão até um a menos do valor da base (n - 1). Para representar o valor da base em qualquer base, você precisa de dois dígitos. Um 0 na posição mais à direita e um 1 na segunda posição representam o valor da própria base. Assim, 10 é dez na base 10, 10 é oito na base 8 e 10 é dezesseis na base 16. Pense nisso. A consistência dos sistemas numéricos é, na verdade, bastante elegante.

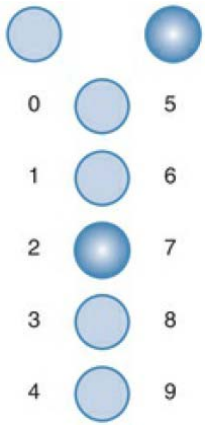
Representação de Números Biquinários

O console do IBM 650, um computador comercial popular no final da década de 1950, permitia ao operador ler o conteúdo da memória usando o sistema biquinário. Esse sistema de representação numérica utiliza sete luzes para representar os 10 dígitos decimais.



Cortesia dos Arquivos Corporativos da IBM, © International Business Machines Corporation

Cada dígito é representado por duas luzes, uma das duas superiores e uma das cinco inferiores. Se a luz superior esquerda estiver acesa, as outras cinco luzes representam 0, 1, 2, 3 e 4, respectivamente, de cima para baixo. Se a luz superior direita estiver acesa, as outras cinco luzes representam 5, 6, 7, 8 e 9. A seguinte configuração representa o número 7:



O IBM 650 era chamado de Ford Tri-Motor dos computadores: assim como o Ford Tri-Motor, os antigos IBM 650 foram enviados para a América Latina, onde tiveram uma vida útil mais longa.

A adição e a subtração de números em outras bases são realizadas exatamente como em números decimais.

Aritmética em Outras Bases

Relembre a ideia básica da aritmética em decimal: 0 + 1 é 1; 1 + 1 é 2; 2 + 1 é 3 e assim por diante. As coisas ficam interessantes quando você tenta somar dois números cuja soma é igual ou maior que o valor da base – por exemplo, 1 + 9. Como não há um símbolo para 10, reutilizamos os mesmos dígitos e dependemos da posição. O dígito mais à direita reverte para 0 e há um transporte para a próxima posição à esquerda. Assim, 1 + 9 é igual a 10 na base 10. As regras da aritmética binária são análogas, mas ficamos sem dígitos muito mais cedo. Ou seja, 0 + 1 é 1 e 1 + 1 é 0 com um transporte. Então, a mesma regra é aplicada a todas as colunas de um número maior, e o processo continua até que não tenhamos mais dígitos para somar. O exemplo abaixo soma os valores binários 101110 e 11011. O valor de transporte é marcado acima de cada coluna com outra cor.

	1	1	1	1	1		← vai-um, ou *carry-out*
		1	0	1	1	1	0
+			1	1	0	1	1
	1	0	0	1	0	0	1

Podemos nos convencer de que esta resposta está correta convertendo ambos os operandos para base 10, somando-os e comparando o resultado: 101110 é 46, 11011 é 27 e a soma é 73. Obviamente, 1001001 é 73 na base 10.

Os fatos de subtração que você aprendeu na escola primária eram que 9 - 1 é 8, 8 - 1 é 7, e assim por diante, até tentar subtrair um dígito maior de um menor, como 0 - 1. Para realizar esse feito, você precisa "pegar emprestado um" do próximo dígito à esquerda do número do qual você está subtraindo. Mais

precisamente, você pega emprestado uma potência da base. Então, na base 10, quando você pega emprestado, você pega emprestado 10. A mesma lógica se aplica à subtração binária. Cada vez que você pega emprestado em uma subtração binária, você pega emprestado 2. Aqui estão dois exemplos com os valores emprestados marcados acima.

```
                  1
            0  ~~2~~  2   ← vem-um
  1  1  1  0  0  1
–          1  1  0
-------------------
  1  1  0  0  1  1
```

```
               0  2
            ~~0~~  2      ← vem-um
  1  1  1  1  0  1
–          1  1  0
-------------------
  1  1  0  1  1  1
```

Mais uma vez, você pode verificar o cálculo convertendo todos os valores para a base 10 e subtraindo para ver se as respostas correspondem.

Sistemas Numéricos de Potência de 2

Números binários e octais compartilham uma relação muito especial: dado um número em binário, você pode lê-lo em octal; Dado um número em octal, você pode lê-lo em binário. Por exemplo, pegue o número octal 754. Se você substituir cada dígito pela representação binária desse dígito, você terá 754 em binário. Ou seja, 7 em octal é 111 em binário, 5 em octal é 101 em binário e 4 em octal é 100 em binário, então 754 em octal é 111101100 em binário.

Para facilitar esse tipo de conversão, a tabela 2.1 abaixo mostra a contagem em binário de 0 a 10 com seus equivalentes octais e decimais.

TABELA 2.1 – Contagem em binário de 0 a 10

Binário	Octal	Decimal
0	0	0
1	1	1
10	2	2
11	3	3
100	4	4
101	5	5
110	6	6
111	7	7
1000	10	8
1001	11	9

? Você consegue contar até três?

Não instintivamente! Psicólogos cognitivos demonstraram que crianças em idade pré-escolar não identificam mais de três conjuntos: um conjunto de um objeto, dois objetos e três ou mais objetos (também chamados de muitos). Até cerca de dois séculos atrás, várias línguas tinham apenas duas ou três palavras numéricas: palavras para único, par e muitos. Ainda temos palavras assim em inglês: *gang, pile, bunch, flock, herd, school, fleet, pride, pack* e *gaggle*.[3]

Para converter de binário para octal, comece no dígito binário mais à direita e marque os dígitos em grupos de três. Em seguida, converta cada grupo de três para seu valor octal.

111	101	100
7	5	4

Vamos converter o número binário 1010110 para octal e, em seguida, converter esse valor octal para decimal. A resposta deve ser o equivalente a 1010110 em decimal, ou 86.

1	010	110
1	2	6

$$
\begin{aligned}
1 * 8^2 &= 1 * 64 = 64 \\
+\ 2 * 8^1 &= 2 * 8 = 16 \\
+\ 6 * 8^0 &= 6 * 1 = 6 \\
&\qquad\qquad\quad 86
\end{aligned}
$$

A razão pela qual o binário pode ser imediatamente convertido para octal e octal para binário é que 8 é uma potência de 2. Existe uma relação semelhante entre binário e hexadecimal. Cada dígito hexadecimal pode ser representado por quatro dígitos binários. Vamos pegar o número binário 1010110 e convertê-lo para hexadecimal marcando os dígitos da direita para a esquerda em grupos de quatro.

101	0110
5	6

$$\begin{aligned} 5 * 16^1 = \quad & 5 * \quad 16 = \quad 80 \\ + 6 * 16^0 = \quad & 6 * \quad 1 = \quad 6 \\ & \qquad\qquad\quad 86 \end{aligned}$$

Agora, vamos converter ABC em hexadecimal para binário. São necessários quatro dígitos binários para representar cada dígito hexadecimal. A em hexadecimal é 10 em decimal e, portanto, é 1010 em binário. Da mesma forma, B em hexadecimal é 1011 em binário, e C em hexadecimal é 1100 em binário. Portanto, ABC em hexadecimal é 101010111100 em binário.

Em vez de confirmar que 101010111100 é 2748 em decimal diretamente, vamos marcá-lo em octal e converter o octal.

101	101	111	100
5	2	7	4

Portanto, 5274 em octal é 2748 em decimal.

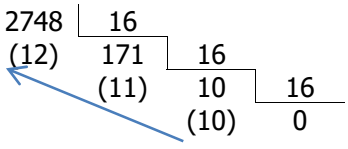
Convertendo da Base 10 para Outras Bases

As regras para converter números de base 10 envolvem a divisão pela base para a qual você está convertendo o número. Dessa divisão, obtém-se um quociente e um resto. O resto se torna o próximo dígito no novo número (da direita para a esquerda), e o quociente substitui o número a ser convertido. O processo continua até que o quociente seja zero. Vamos escrever as regras de uma forma diferente.

```
WHILE (o quociente não é zero)
   Divida o número decimal pela nova base
   Faça do resto o próximo dígito à esquerda na resposta
   Substitua o número decimal pelo quociente
```

Essas regras formam um *algoritmo* para conversão da base 10 para outra base. Um algoritmo é uma sequência lógica de etapas que resolve um problema. Temos muito mais a dizer sobre algoritmos nos capítulos seguintes. Aqui, mostramos uma maneira de descrever um algoritmo e em seguida, aplicá-lo para realizar as conversões.

A primeira linha do algoritmo nos diz para repetir as próximas três linhas até que o quociente da nossa divisão se torne zero. Vamos converter o número decimal 2748 para hexadecimal. Como vimos em exemplos anteriores, a resposta deve ser ABC.



O resto (12) é o primeiro dígito da resposta hexadecimal, representado pelo dígito C. Portanto, a resposta até agora é C. Como o quociente não é zero, nós o dividimos (171) pela nova base (16).

O resto (11) é o próximo dígito à esquerda na resposta, representado pelo dígito B. Agora, a resposta até agora é BC. Como o quociente não é zero, dividimos (10) pela nova base.

O resto (10) é o próximo dígito à esquerda na resposta, representado pelo dígito A. Agora a resposta é ABC. O quociente é zero, então terminamos, e a resposta final é ABC.

Valores Binários e Computadores

Embora alguns dos primeiros computadores fossem máquinas decimais, os computadores modernos são máquinas binárias. Ou seja, os números dentro do computador são representados em formato binário. Na verdade, todas as informações são de alguma forma representadas usando valores binários. O motivo é que cada local de armazenamento dentro de um computador contém um sinal de baixa voltagem ou um sinal de alta voltagem. Como cada local pode ter apenas um de dois estados, é lógico igualar esses estados a 0 e 1. Um sinal de baixa tensão é igualado a 0, e um sinal de alta tensão é igualado a 1. Na verdade, você pode esquecer as tensões e pensar em cada local de armazenamento como contendo 0 ou 1. Observe que um local de armazenamento não pode estar vazio: ele deve conter 0 ou 1.

Grace Murray Hopper



© Cynthia Johnson/Getty Images

De 1943 até sua morte no dia de Ano Novo de 1992, a Almirante Grace Murray Hopper esteve intimamente envolvida com a computação. Em 1991, ela foi condecorada com a Medalha Nacional de Tecnologia "por suas realizações pioneiras no desenvolvimento de linguagens de programação que simplificaram a tecnologia da computação e abriram as portas para um universo significativamente maior de usuários".

A Almirante Hopper nasceu Grace Brewster Murray na cidade de Nova York em 9 de dezembro de 1906. Ela estudou em Vassar e obteve um doutorado em matemática em Yale. Nos 10 anos seguintes, lecionou matemática em Vassar.

Em 1943, a Almirante Hopper ingressou na Marinha dos EUA e foi designada para o Projeto de Computação do Bureau of Ordnance na Universidade de Harvard como programadora no Mark I. Após a guerra, ela permaneceu em Harvard como docente e continuou a trabalhar nos computadores Mark II e Mark III da Marinha. Ela adorava contar a história de como, enquanto trabalhava no Mark II, um dos operadores descobriu o primeiro "bug" de computador – uma mariposa presa em um dos relés. Em 1949, ela ingressou na Eckert-Mauchly Computer Corporation e trabalhou no UNIVAC I.

A Almirante Hopper tinha um compilador funcional em 1952, uma época em que a sabedoria convencional era de que os computadores só podiam fazer cálculos aritméticos. Embora não fizesse parte do comitê que projetou a linguagem de computador COBOL, ela foi ativa em seu projeto, implementação e uso. COBOL (sigla para Common Business-Oriented Language) foi desenvolvido no início da década de 1960 e ainda é amplamente utilizado no processamento de dados empresariais. A Almirante Hopper aposentou-se da Marinha em 1966, apenas para ser reconvocada um ano depois para o serviço ativo em tempo integral. Sua missão era supervisionar os esforços da Marinha para manter a uniformidade nas linguagens de programação. Diz-se que, assim como o Almirante Hyman Rickover foi o pai da marinha nuclear, a Contra-Almirante Hopper foi a mãe da automação de dados computadorizada na Marinha. Ela serviu no Comando de Automação de Dados Navais até se aposentar novamente em 1986 com a patente de Contra-Almirante. Na época de sua morte, ela era consultora sênior na Digital Equipment Corporation.

A Almirante Hopper amava os jovens e gostava de dar palestras em campi universitários. Ela frequentemente distribuía fios coloridos, que chamava de nanossegundos, porque eram cortados em um comprimento de cerca de 30 centímetros – a distância que a luz percorre em um nanossegundo (bilionésimo de segundo). Seu conselho aos jovens era: "Vocês gerenciam as coisas, vocês lideram as pessoas. Exageramos na gestão e esquecemos da liderança."

Ao longo de sua vida, a Almirante Hopper recebeu títulos honorários de mais de 40 faculdades e universidades. Ela foi homenageada por seus pares em diversas ocasiões, incluindo o primeiro prêmio de Homem do Ano em Ciências da Computação, concedido pela Data Processing Management Association – Associação de Gerenciamento de Processamento de Dados, e o Prêmio de Contribuidores para a Educação em Ciência da Computação, concedido pelo Special Interest Group for Computer Science Education (SIGCSE) – Grupo de Interesse Especial para Educação em Ciência da Computação, que faz parte da Association for Computing Machinery (ACM) – Associação para Máquinas de Computação.

Nell Dale, ao notificar a Almirante Hopper sobre o prêmio SIGCSE, perguntou de qual de suas muitas realizações ela mais se orgulhava. Ela respondeu: "Todos os jovens que treinei ao longo dos anos."

Cada unidade de armazenamento é chamada de digito binário – binary digit ou bit, abreviadamente. Os bits são agrupados em bytes (8 bits) e os bytes são agrupados em unidades chamadas words – ou palavras. O número de bits em uma palavra é conhecido como o tamanho da palavra do computador. Por exemplo, a arquitetura do IBM 370 no final da década de 1970 tinha meias palavras (2 bytes ou 16 bits), palavras inteiras (4 bytes) e palavras duplas (8 bytes).

- Dígito binário: Um dígito no sistema numérico binário; um 0 ou um 1.
- Bit: Dígito binário.
- Byte: Conjunto de oito dígitos binários.
- Palavra: Um grupo de um ou mais bytes; o número de bits em uma palavra é o tamanho da palavra do computador.

Na época da edição deste livro, os computadores eram frequentemente máquinas de 32 bits (como o processador Pentium IV da Intel) ou máquinas de 64 bits (como os processadores Alpha da Hewlett-Packard e o processador Itanium 2 da Intel). No entanto, alguns microprocessadores usados em aplicações como pagers são máquinas de 8 bits. A máquina de computação que você está usando – seja ela qual for – é, em última análise, suportada pelo sistema numérico binário.

Temos muito mais a explorar sobre a relação entre computadores e números binários. No próximo capítulo, examinaremos muitos tipos de dados e veremos como eles são representados em um computador. No Capítulo 4, veremos como controlar sinais elétricos que representam valores binários. No Capítulo 6, veremos como os números binários são usados para representar comandos de programa que o computador executa.

RESUMO

Os números são escritos usando a notação posicional, na qual os dígitos são organizados em sucessão, a posição de cada dígito tem um valor posicional e o número é igual à soma dos produtos de cada dígito por seu valor posicional. Os valores posicionais são potências da base do sistema numérico. Assim, no sistema numérico decimal, os valores posicionais são potências de 10; no sistema numérico binário, os valores posicionais são potências de 2.

A aritmética pode ser realizada com números em qualquer base representada em notação posicional. As mesmas regras operacionais se aplicam a outras bases, assim como à base 10. Adicionar 1 ao maior dígito da base causa um transporte para a próxima posição.

Base 2, base 8 e base 16 estão todas relacionadas porque essas bases são potências de 2. Essa relação fornece uma maneira rápida de converter números nessas bases. O hardware do computador é projetado usando números na base 2. Um sinal de baixa tensão é igual a 0, e um sinal de alta tensão é igual a 1.

EXERCÍCIOS

Para os Exercícios 1 a 5, relacione os seguintes números com suas definições.

A. Número B. Número natural C. Número inteiro D. Número negativo E. Número racional

1. Uma unidade de um sistema matemático abstrato sujeito às leis da aritmética.
2. Um número natural, um negativo de um número natural ou zero.
3. O número zero e qualquer número obtido pela adição repetida de um a ele.
4. Um número inteiro ou o quociente de dois números inteiros (excluindo a divisão por zero).
5. Um valor menor que zero, com sinal oposto ao seu equivalente positivo.

Para os exercícios 6 a 11, combine a solução com o problema.

A. 10001100 B. 10011110 C. 1101010 D. 1100000 E. 1010001 F. 1111000

6. 1110011 + 11001 (adição binária)
7. 1010101 + 10101 (adição binária)
8. 1111111 + 11111 (adição binária)
9. 1111111 – 111 (subtração binária)

10. 1100111 – 111 (subtração binária)
11. 1010110 – 101 (subtração binária)

Para os Exercícios 12 a 17, marque as respostas como verdadeiras ou falsas, da seguinte forma:

A. Verdadeiro B. Falso

12. Os números binários são importantes em computação porque um número binário pode ser convertido em qualquer outra base.
13. Números binários podem ser lidos em hexadecimal, mas não em octal.
14. Começando da esquerda para a direita, cada agrupamento de quatro dígitos binários pode ser lido como um dígito hexadecimal.
15. Um byte é composto por seis dígitos binários.
16. Dois dígitos hexadecimais não podem ser armazenados em um byte.
17. Ler dígitos octais como binários produz o mesmo resultado, seja lido da direita para a esquerda ou da esquerda para a direita.

Os exercícios 18 a 47 são problemas ou perguntas de resposta curta.

18. Distinga entre um número natural e um número negativo.
19. Distinga entre um número natural e um número racional.
20. Identifique os seguintes números como naturais, negativos ou racionais.
    a) 1,333333 b) –1/3 c) 1066 d) 2/5 e) 6,2 f) π (pi)
21. Quantas unidades existem em 891 se ele for um número em cada uma das seguintes bases?
    a) Base 10 b) Base 8 c) Base 12 d) Base 13 e) Base 16
22. Expresse o número 891 como um polinômio em cada uma das bases do Exercício 21.
23. Converta os seguintes números da base mostrada para a base 10.
    a) 111 (base 2) b) 777 (base 8) c) FEC (base 16) d) 777 (base 16) e) 111 (base 8)
24. Explique como a base 2 e a base 8 estão relacionadas.
25. Explique como a base 8 e a base 16 estão relacionadas.
26. Expanda a tabela 2.1 para incluir os decimais de 11 a 16.
27. Expanda a tabela do Exercício 26 para incluir números hexadecimais.
28. Converta os seguintes números binários para octal.
    a) 111110110 b) 1000001 c) 10000010 d) 1100010
29. Converta os seguintes números binários para hexadecimal.
    a) 10101001 b) 11100111 c) 01101110 d) 01121111
30. Converta os seguintes números hexadecimais para octal.
    a) A9 b) E7 c) 6E d) CAFE e) FACA
31. Converta os seguintes números octais para hexadecimal.
    a) 777 b) 605 c) 443 d) 521 e) 1
32. Converta os seguintes números decimais para octal.
    a) 901 b) 321 c) 1492 d) 1066 e) 2001
33. Converta os seguintes números decimais para binário.
    a) 45 b) 69 c) 1066 d) 99 e) 1

34. Converta os seguintes números decimais para hexadecimais.
    a) 1066 b) 1939 c) 1 d) 998 e) 43

35. Se você fosse representar números na base 18, quais símbolos usaria para representar os números decimais de 10 a 17, além de letras?

36. Converta os seguintes números decimais para a base 18 usando os símbolos que você sugeriu no Exercício 35.
    a) 1066 b) 99099 c) 1

37. Execute as seguintes adições octais.
    a) 770 + 665 b) 101 + 707 c) 202 + 667

38. Execute as seguintes adições hexadecimais.
    a) 19AB6 + 43 b) AE9 + F c) 1066 + ABCD

39. Execute as seguintes subtrações octais.
    a) 1066 – 776 b) 1234 – 765 c) 7766 – 5544

40. Execute as seguintes subtrações hexadecimais.
    a) ABC – 111 b) 9988 – AB c) A9F8 – 1492

41. Por que os números binários são importantes na computação?
42. Quantos bits um byte contém?
43. Quantos bytes existem na palavra em uma máquina de 64 bits?
44. Por que microprocessadores, como pagers, possuem apenas palavras de 8 bits?
45. Por que é importante estudar como manipular números de tamanho fixo?
46. Quantas unidades existem no número AB98 na base 13?
47. Descreva como funciona uma representação numérica biquinária.

QUESTÕES DE REFLEXÃO

1. O Exercício 20 pediu que você classificasse π como uma das opções. O π não pertence a nenhuma das categorias nomeadas; π (e e) são números transcendentais. Procure números transcendentais no dicionário ou em um livro de matemática antigo e dê a definição com suas próprias palavras.

2. Números complexos são outra categoria de números que não é discutida neste capítulo. Procure números complexos em um dicionário ou em um livro antigo de matemática e dê a definição com suas próprias palavras.

3. Muitas ocorrências cotidianas podem ser representadas como um bit binário. Por exemplo, uma porta está aberta ou fechada, o fogão está ligado ou desligado e o cachorro está dormindo ou acordado. Relacionamentos poderiam ser representados como um valor binário? Discuta a questão, dando exemplos.

4. Você já tinha ouvido falar do Tribunal FISA antes de ler este capítulo? Agora você tem uma melhor compreensão do que ele é?